

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI (RAJ.)
Second Semester 2019-20 CS F111 Computer Programming
Comprehensive Exam (Open-book)

Date: 7th Jan 2020

Weightage: 45%

Max. Marks: 90

Duration: 2 hrs.

Note:

1. The test is open book in nature and you are allowed to use any resource (book, eBook, google search, etc.). Some questions use certain concepts which have not been explicitly taught in the class, but the related or underlying concept has surely been taught. Given the open book nature of the component, this is intentional, and it is expected that you will explore those concepts (through any resource you wish) and then work out your answers.
2. Recommended time for PART A is 45 minutes. Max. allowed time is 60 minutes. You can ask for PART B after a minimum of 30 minutes.
3. Write your answers in the Google Form provided for PART A.
4. For all C codes, assume that the required header files are present.

Write the output of the following code snippets or write the error (compile time/ run time etc) with brief reasoning. Note: reasoning required only in case of error. **[15*2 = 30 M]**

1	<pre>int a= -99, b= -1, c = 0; int d = 10, r1, r2; r1= ++a && ++b && ++c && ++d; r2= ++a ++b ++c ++d; printf("%d %d", r1,r2);</pre>	2	<pre>const char var = 'J'; ++var; printf("var is %f", var);</pre>	3	<pre>int v1= 99, v2= -99; if (v1-v2 < 0) v2 = v1 * v2; else break; printf ("%d", v2);</pre>
4	<pre>int a[5] = {1,2,3,4,5}; int *p = a+2; for (int i=0;i<3;i++) printf("%d", p[-i]);</pre>	5	<pre>int n=512; static int c = n; while (n){ c++; n /= 2; } (c!=0)? printf ("%d", ++c): printf ("%d", --c);</pre>	6	<pre>int i = 9; int j = ++i++; printf("j is %d", j);</pre>
7	<pre>static int var; int new = sizeof(var++) + ++var; printf("%d %d", var, new); //assume int as 4 bytes</pre>	8	<pre>char* str ="CSF111"; str[6] = 'A'; printf("%s", str);</pre>	9	<pre>int x = ((4,5),3,(8,0)); int y = 8/4%3+5-3*4>=8; printf("%d %d", x,y);</pre>
10	<pre>int *ptr,arr[2][2]= {1,2,3,4}; ptr = *arr+1 ; printf("%d ", *(ptr));</pre>	11	<pre>static int var; char c = -2; printf("%d",--var+ ~c);</pre>	12	<pre>int i=10; static int x=i; if(x==i) printf("One"); else printf("Two");</pre>
13	<pre>for(int i=1,j=9;;) switch(i&&j) { default: printf("Yes"); case 1: printf("No"); case 0: printf("Good"); }</pre>	14	<pre>int x; float y; x= 3,17,49; y=(3.5,7.1,4.5); printf("%f", x+y);</pre>	15	<pre>int a=0, b=0, x; x = 121 (a = ++b); printf("%d %d %d\n", a, b, x);</pre>

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI (RAJ.)
Second Semester 2019-20 CS F111 Computer Programming
Comprehensive Exam (Open-book)

Date: 7th Jan 2020

Weightage: 45%

Max. Marks: 90

Duration: 2 hrs.

Note:

1. The test is open book in nature and you are allowed to use any resource (book, eBook, google search, etc.). Some questions use certain concepts which have not been explicitly taught in the class, but the related or underlying concept has surely been taught. Given the open book nature of the component, this is intentional, and it is expected that you will explore those concepts (through any resource you wish) and then work out your answers.
 2. This is PART B worth 60 marks. You should solve this after submitting PART A.
 3. Write your answers in the Google Form provided for PART B.
 4. For all C codes, assume that the required header files are present.
-
1. The below program takes student records from a text file “Records.txt”. Total five student records are provided in Records.txt, each containing name, roll number, marks and a bonus field. These fields are separated by a colon between them. The contents of the file Records.txt are shown below:

Records.txt

George Erwin:1:78:0
Markus Clinton:12:65:1
Bill Cook:13:80:1
Albert Elastir:14:85:0
Steve Dewot:15:62:1

Based on the bonus field (the last entry in each row), the student records are classified. If the bonus field is marked as 1, the student record is written in a separate file “Modified_marks.txt”. Otherwise, another separate file “No_change.txt” is populated. The overall intention of the code is to classify the modified/non-modified records by writing them in the corresponding separate files “Modified_marks” and “No_change”. The code to accomplish this task is provided below. The filenames Records.txt, Modified_marks.txt and No_change.txt are supplied as command line arguments to the program.

Functions used:

fgets: This function reads a number of characters from a given file stream source into an array of characters.

sscanf: It reads the formatted input from a string. In the above program, it is used to read the number of records in each line from Info.txt so that they can be further checked for valid or invalid.

fprintf: This function is same as printf() but instead of writing data to the console, it writes formatted data to the file.

```

1.  #include<stdio.h>
2.  #include<stdlib.h>
3.  int main(int argc, char **argv)
4.  {
5.      FILE *fp1, *fp2, *fp3;
6.      short m1, m2, m3, items;
7.      char buf[80], name[20];
8.      if(argc!=4){
9.          fprintf(stderr, "Usage: %s invalid filename", argv[0]);
10.         exit(1);
11.     }
12.     else if((fp1 = fopen(argv[1], "r")) == NULL || (fp2 = fopen(argv[2],
13.         "w")) == NULL || (fp3 = fopen(argv[3], "w")) == NULL){
14.         printf("File permission denied");
15.         exit(0);
16.     }
17.     while(fgets(buf, sizeof(buf), fp1) != NULL){
18.         items = sscanf(buf, "%[^:]:%hd:%hd:%hd", name, &m1, &m2, &m3);
19.         if(__(A))
20.             fputs(__(B), fp2);
21.         else
22.             fprintf(__(C), "%s:%hd:%hd:%hd\n", name, m1, m2, m3);
23.     }
24.     exit(0);

```

Attempt the following questions and precisely write your answer. **[6 + 4 + 5 +3 = 18M]**

- a) Fill in the blank spaces (quoted as (A), (B), and (C)) in the program to make it functioning.

Without re-writing the entire program:

- b) Provide a sample code to create a backup of the file Records.txt by copying its content to another file Data.txt.
- c) Write the output of the following code when written for the “Records.txt” file (represented by *fp1 in the program):

```

int k;
fseek(fp1, 1L, SEEK_END);
while(fseek(fp1, -2L, SEEK_CUR) == 0)
    if ((k = fgetc(fp1)) != EOF)
        putchar(k);

```

- d) In a single statement, write the task done by the following code when written for the “No_change.txt” file (represented by *fp3 in the program):

```

struct data{
    char c1[50];
    char c2[50];
    char c3[50];

```

```

} d[2] = {
    {"Birla:Science:Centre:Pilani"},
    {"Computer:Programming:using:C"}
};
int size = sizeof(struct data);
fwrite(&d[0], size, 2, fp3);

```

2. Read the following algorithm carefully and then write a C program to find the approximate root **r** to the equation $x^3 - 2x - 5 = 0$. Use the float data type wherever you need floating point numbers. Use appropriate C libraries as required. Add comments as appropriate. **[15M]**

Algorithm:

Input:	$f: R \rightarrow R$ //continuous function that changes sign on interval (a,b) Epsilon // Degree of accuracy
Output:	r // approximate root repeat $c = \frac{af(b)-bf(a)}{f(b)-f(a)}$ if $f(c) = 0$ or $ f(c) \leq \text{Epsilon}$ then r =c return r end if $f(a).f(c) < 0$ then b = c else a = c end end

3. We want to make a text processor in C that does the following **[15M]**
- Accumulate an input from keyboard into a buffer of 512 characters and that can be terminated by EOF. The program should accept only the characters that are useful to construct a text i.e. alphanumeric, punctuations and new line. If it detects any control character input (that may happen by mistyping), it notifies by Bell signal.
 - At the end of keyboard input, print the text size, number of punctuations and number of alphanumeric characters in the accumulated text.

Your code MUST (compulsorily) make use of relevant C libraries `isalnum()`, `ispunct()` and `isctrl()` available in `ctype.h`. Use the code given below to get started.

```

# include <stdio.h>
#include <ctype.h>
# define BELL          07 /* Bell character */
#define BUF_size 512
int c;
char buf[BUF_size];
main(){
    // your code goes here
}

```

4. Consider a structure, employee, containing name, salary and credit_point as member fields. An array of structure is used to store the overall details for a group of employees. The below program is meant to compute the appraisal for given record of employees. If the credit_point is set, the respective employee is awarded with appraisal that increments his/her salary by 10%. On the contrary, if the credit_point is zero, the salary remains intact without any modification. To perform this task, a separate function “appraisal” is provided. Once the appraisal is done, the employee names who are awarded and their modified salaries should be printed on the screen. *Without adding any new variable*, you are required to write the appropriate missing codes in the program so that its overall functioning can be achieved. **[12M]**

```

1. #include<stdio.h>
2. #define cols 5
3. struct employee{
4.     char name[30];
5.     int salary;
6.     int credit_point;
7. } e[cols] = {{ "Andrew", 3000, 0},
8.             {"Mark", 2500, 1},
9.             {"Reinhard", 5000, 1},
10.            {"Adleman", 4000, 0},
11.            {"Pei", 3500, 1}};
12. struct employee appraisal(struct employee);
13. int main(){
14.     int i;
15.     printf("Employees who are awarded with appraisal and their modified
        i. salary\n");
16.     for(____MISSING CODE (A)____){
17.         ____MISSING CODE (B)____
18.     }
19. return 0;
20. }
21. struct employee appraisal(struct employee e1){
22.     ____MISSING CODE (C)____
23. }

```